La naturaleza es el tesoro más preciado.

HW05 - Object-Oriented Modeling and UML: Summary

1. OS.01 - Object Orientation Good Practices

1. Conceptual Modeling

Conceptual modeling represents real-world systems to understand problems, communicate ideas, document decisions, and guide software development.

2. UML Diagrams

- Use Case Diagrams: Describe users and their goals.
- Class Diagrams: Show classes and their relationships.
- Interaction Diagrams: Show communication between objects.
- State Charts: Describe objects behavior.

3. Good Programming Practices

- Organize software into loosely coupled layers.
- Prefer aggregation over inheritance when appropiate.
- Program to interfaces or abstractions.
- Use meaning full names for classes, variables and methods.
- Apply unit testing to verify correctness and detect errors.

1. OS.07 - Introductions to UML

1. What is UML?

UML (Unified Modeling Language) is an object-oriented modeling language used to visualize, specify and document software systems.

2. Main Characteristics

- Expressive and semi-formal.
- Supports incremental development.
- Independent of development process.
- Customizable and extensible.

3. UML Building Blocks

UML includes structural, behavioral, organizational, and annotational elements.

4. Main UML Diagrams

- Class Diagram: Shows classes and relationships.
- Object Diagram: Represents objects instances.
- Use Case Diagram: Describes actors and goals.
- Sequence Diagram: Shows interactions over time.
- Activity Diagram: Represents workflows.
- State Diagram: Shows states and transitions.

5. Modeling Perspectives.

- Analysis: Describes the problem domain.
- Specifications: Defines the logical software system.
- Implementation: Describes software components and design.

1.05.08 - Basic Use Cases

1. Actors.

Actors are external entities that interact with a system, including users, other systems, or hardware.

2. Use Cases

Use cases describe what users need to accomplish with a system. They focus on user goals rather than implementation details.

3. Use Case Diagrams

They visually represents systems functionality through four elements:

- Actors
- Use Cases
- Relationships.
- Subjects.

4. Use Case Relationships

- <<include>>: A required sub-goal of another use case.
- <<extend>>: Additional behavior under special conditions.
- Generalizations: A specialized use case inherits characteristics from a general one.

5. Use cases vs. System Interactions.

- Use Cases: Explain what users want and why.
- System interactions: Explain how users interact with the system.

Use case diagrams help identify requirements and improve communication.

La naturaleza es el tesoro más preciado.

1.05.09 - Objects, Classes, and Basic Class Diagrams.

1. Objects.

Objects are fundamental elements of object-oriented systems. Each object has identity, state, and behavior.

2. Classes.

A class represents a collections of objects sharing common characteristics and behaviors.

3. Class Structure.

A UML class can contain:

- Name: Identifies the class.
- Attributes: Define object properties.
- Operations: Represent object behavior.

4. Class Relationships

- Association: Connects related classes.
- Generalizations: Represents inheritance.
- Aggregation: Describes a whole - part relationship.
- Composition: Strong aggregation with dependent lifetimes.
- Dependency: One class uses another.

5. Multiplicity

Multiplicity specifies how many objects can participate in a relationship.

Examples:

- 1: Exactly one.
- 0..1: Zero or one.
- 0..*: Zero or many.
- 1..*: One or many.

6. Class Diagram Purpose.

Class diagrams describe the structure of a system, including its classes, attributes, operations, and relationships.

La naturaleza es el tesoro más preciado.

1. 05. 10 - UML Quick Reference Guide.

1- UML Best Practices

- Model real-world problems during early stages.
- Keep diagrams simple and readable.
- Avoid unnecessary line crossings.
- Focus each diagram on a specific idea.
- Use appropriate UML diagrams.

2. Use Case Diagrams.

Represents actors, goals, and relationships such as <<include>>, <<extend>>, and generalization.

3. Class Diagrams.

Describes classes and their relationships using associations, aggregation, composition, generalization, and multiplicity.

4. Activity Diagrams

Represents workflows, activities, decisions, and parallel processes.

5- Interactions Diagrams.

- Sequence Diagrams: show messages exchanged over time.
- Communication Diagrams: show interactions between connected objects.

6. State Charts

Describe the states of an objects and transitions caused by events.

7. General Recommendations.

Choose the appropriate diagram, use clear names, and represent only the information necessary to communicate the systems designs.